

Civic Edition

VOL. 1, NO. 058

2026-03-30

FEATURES

Emulating Ruby's Benchmark.real_time in Python Using a Context Manager

Scott Johnson (fuzzyblog)

One of the most confusing aspects of multi language programming is when you know how to do it in your preferred language and when you are trying to figure it out in your new language. And, as any reader of this blog knows, Ruby is always, always, always my preferred language. But, as my perspective on languages widens, I suspect you could say that:

Ruby is my wife but

I have a serious relationship with Python

And I'm now flirting with Rust and seeing if I want to bring her to the dance ...

But enough of that foolishness, today's topic is code benchmarking. Here's what I would do in Ruby:

```
benchmarks = {} results = nil elapsed_time = Benchmark . realtime do results = some_long_running_method ( foo , bar , baz ) end benchmarks [ 'some_long_running_method' ] = elapsed_time
```

By defining the results variable outside the block of code (the bits between do and end), it basically gets a transitive scope that allows what is computed inside the block to exist outside the block. And then I set the associative array (also known as a hash) to have a key for the method, some_long_running_method, and also the elapsed time. This way I can build up a hash of all my methods and how long they take.

Note : A block is a “lexical scope” during which (the code between the do and end), variables are local only to that scope (unless you “shadow” it with a variable of the same name).

My pairing partner on this project, Grant, has told me “You could use a context manager for this”. And this led me to these Python docs:

ContextLib

Geeks for Geeks

I found both of these very pythonic – which, for me, means I didn't understand them at all . Happily I was able to cobble together a basic example:

```
from contextlib import contextmanager import time

def some_long_running_method1 ( amount ): print ( amount )

def some_long_running_method2 ( amount ): print ( amount )
```

```
@ contextmanager def timer ( benchmarks , benchmark_key ): t_0 = time . time () yield run_time = time . time () - t_0 benchmarks [ benchmark_key ] = run_time
```

```
benchmarks = {}
```

```
with timer ( benchmarks , 'test_sleep' ): some_long_running_method1 ( 5 )
```

```
with timer ( benchmarks , 'test_foo' ): some_long_running_method1 ( 5 )
```

Note : The @contextmanager is apparently a “python decorator”. I don't really understand decorators any more than I understand interior decorators so, well, there's that.

The end result of this is a Python dictionary like this:

```
{ 'some_long_running_method1' : 4.696846008300781e-05 , 'some_long_running_method2' : 4.696846008300781e-05 }
```

And here's how this looks like in a production code refactor:

```
times = {}
```

```
time_start = time . time () json_str = common_kafka . pop_from_topic ( kafka_conn , kafka_topic_name ) time_end = time . time () times = common_debugging . benchmark ( times , time_start , time_end , 'common_kafka.pop_from_topic' )
```

You should note that my timer routine now is located in a common file named common_debugger.py.

```
import common_debugger
```

```
times = {} with common_debugging . timer ( times , 'common_kafka.pop_from_topic' ): json_str = common_kafka . pop_from_topic ( kafka_conn , kafka_topic_name )
```

This is pretty clearly an improvement in terms of reducing complexity and code cruft. Context managers are an interesting improvement to Python even if I personally found the documentation confusing. Recommended .

Instagram and Facebook deliver instant playback and boost user engagement with Media3 PreloadManager

Android Developers · Android Developers Blog

In the dynamic world of social media, user attention is won or lost quickly. Meta apps (Facebook and Instagram) are among the world's largest social platforms and serve billions of users globally. For Meta, delivering videos seamlessly isn't just a feature, it's the core of their user experience. Short-form videos, particularly Facebook Newsfeed and Instagram Reels, have become a primary driver of engagement. They enable creative expression and rapid content consumption; connecting and entertaining people around the world.

This blog post takes you through the journey of how Meta transformed video playback for billions by delivering true instant playback.

The latency gap in short form videos

Short-form videos lead to highly fast paced interactions as users quickly scroll through their feeds. Delivering a seamless transition between videos in an ever-changing feed introduces unique hurdles for instantaneous playback. Hence we need solutions that go beyond traditional disk caching and standard reactive playback strategies.

The path forward with Media3 PreloadManager

To address the shifts in consumption habits from rise in short form content and the limitations of traditional long form playback architecture, Jetpack Media3 introduced PreloadManager . This component allows developers to move beyond disk caching, offering granular control and customization to keep media ready in memory before the user hits play. Read this blog series to understand technical details about media playback with PreloadManager.

How Meta achieved true instant playback

Previously, Meta used a combination of warmup (to get players ready) and prefetch (to cache content on disk) for video delivery. While these methods helped improve network efficiency, they introduced significant challenges. Warmup required instantiating multiple player instances sequentially, which consumed significant memory and limited preloading to only a few videos. This high resource demand meant that a more scalable robust solution could be applied to deliver the instant playback expected on modern, fast-scrolling social feeds.

Integrating Media3 PreloadManager

To achieve truly instant playback, Meta's Media Foundation Client team integrated the Jetpack Media3 PreloadManager into Facebook and Instagram. They chose the DefaultPreloadManager to unify their preloading and playback systems. This integration required refactoring Meta's existing architecture to enable efficient resource sharing between the PreloadManager and ExoPlayer instances. This strategic

shift provided a key architectural advantage: the ability to parallelize preloading tasks and manage many videos using a single player instance. This dramatically increased preloading capacity while eliminating the high memory complexities of their previous approach.

Optimization and Performance Tuning

The team then performed extensive testing and iterations to optimize performance across Meta's diverse global device ecosystem. Initial aggressive preloading sometimes caused issues, including increased memory usage and scroll performance slowdowns. To solve this, they fine-tuned the implementation by using careful memory measurements, considering device fragmentation, and tailoring the system to specific UI patterns.

Fine tuning implementation to specific UI patterns

Meta applied different preloading strategies and tailored the behavior to match the specific UI patterns of each app:

Facebook Newsfeed : The UI prioritizes the video currently coming into view. The manager preloads only the current video to ensure it starts the moment the user pauses their scroll. This “ current-only “ focus minimizes data and memory footprints in an environment where users may see many static posts between videos. While the system is presently designed to preload just the video in view, it can be adjusted to also preload upcoming (future) videos.

Instagram Reels : This is a pure video environment where users swipe vertically. For this UI, the team implemented an “ adjacent preload “ strategy. The PreloadManager keeps the videos immediately after the current Reel ready in memory. This bi-directional approach ensures that whether a user swipes up or down, the transition remains instant and smooth. The result was a dramatic improvement in the Quality of Experience (QoE) including improvements in Playback Start and Time to First Frame for the user.

Scaling for a diverse global device ecosystem

Scaling a high-performance video stack across billions of devices requires more than just aggressive preloading; it requires intelligence. Meta faced initial challenges with memory pressure and scroll lag, particularly on mid-to-low-end hardware. To solve this, they built a Device Stress Detection system around the Media3 implementation. The apps now monitor I/O and CPU signals in real-time. If a device is under heavy load, preloading is paused to prioritize UI responsiveness.

This device-aware optimization ensures that the benefit of instant playback doesn't come at the cost of system stability, allowing even users on older hardware to experience a smoother, uninterrupted feed.

Architectural wins and code health

Contact Picker: Privacy-First Contact Sharing

Android Developers · Android Developers Blog

Privacy and user control remain at the heart of the Android experience. Just as the photo picker made media sharing secure and easy to implement, we are now bringing that same level of privacy, simplicity, and great user experience to contact selection.

A New Standard for Contact Privacy

Historically, applications requiring access to a specific user's contacts relied on the broad `READ_CONTACTS` permission. While functional, this approach often granted apps more data than necessary. The new Android Contact Picker, introduced in Android 17, changes this dynamic by providing a standardized, secure, and searchable interface for contact selection.

This feature allows users to grant apps access only to the specific contacts they choose, aligning with Android's commitment to data transparency and minimized permission footprints.

Developers can integrate the Contact Picker using the `Intent.ACTION_PICK_CONTACTS` intent. This updated API offers several powerful capabilities:

Granular Data Requests: Apps can specify exactly which fields they need, such as phone numbers or email addresses, rather than receiving the entire contact record.

Multi-Selection Support: The picker supports both single and multiple contact selections, giving developers more flexibility for features like group invitations.

Selection Limits: Developers can set custom limits on the number of contacts a user can select at one time.

Temporary Access: Upon selection, the system returns a Session URI that provides temporary read access to the requested data, ensuring that access does not persist longer than necessary.

Access to other profiles: When using this new intent, the interface will allow users to select contents from other user profiles such as a work profile, cloned profile or a private space.

Optimized Performance: The Contact Picker returns a single `Uri` that allows for collective result querying, eliminating the need to query individual contact `Uri` separately as required by `ACTION_PICK`. This efficiency further reduces system overhead by utilizing a single `Binder` transaction.

Backward Compatibility and Implementation

For devices running Android 17 or higher, the system automatically upgrades legacy `ACTION_PICK` intents that specify contact data types to the new, more secure interface. However, to take full advantage of advanced features

like multi-selection, developers are encouraged to update their implementation code and utilize the `ContentResolver` to query the returned Session URI.

Integrate the contact picker To integrate the Contact Picker, developers use the `ACTION_PICK_CONTACTS` intent. Below is a code example demonstrating how to launch the picker and request specific data fields, such as email and phone numbers.

```
// State to hold the list of selected contacts
var contacts by remember { mutableStateOf >(emptyList()) }
```

```
// Launcher for the Contact Picker intent
val pickContact = rememberLauncherForActivityResult(StartActivityForResult()) { if (it.resultCode == Activity.RESULT_OK) { val resultUri = it.data?.data ?: return@rememberLauncherForActivityResult }
```

```
// Process the result URI in a background thread
coroutine.launch { contacts = processContactPickerResultUri(resultUri, context) } }
```

```
// Define the specific contact data fields you need
val requestedFields = arrayListOf( Email.CONTENT_ITEM_TYPE, Phone.CONTENT_ITEM_TYPE, )
```

```
// Set up the intent for the Contact Picker
val pickContactIntent = Intent(ACTION_PICK_CONTACTS).apply { putExtra(EXTRA_PICK_CONTACTS_SELECTION_LIMIT, 5) putStringArrayListExtra(EXTRA_PICK_CONTACTS_REQUESTED_DATA, requestedFields) putExtra(EXTRA_PICK_CONTACTS_MATCH_ALL, false) }
```

```
// Launch the picker
pickContact.launch(pickContactIntent)
```

After the user makes a selection, the app processes the result by querying the returned Session URI to extract the requested contact information.

```
// Data class representing a parsed Contact with selected details
data class Contact(val id: String, val name: String, val email: String?, val phone: String?)
```

```
// Helper function to query the content resolver with the URI returned by the Contact Picker.
// Parses the cursor to extract contact details such as name, email, and phone number
private suspend fun processContactPickerResultUri( sessionUri: Uri, context: Context ): List = withContext(Dispatchers.IO) { // Define the columns we want to retrieve from the ContactPicker ContentProvider
val projection = arrayOf( ContactsContract.Contacts._ID, ContactsContract.Contacts.DISPLAY_NAME_PRIMARY, ContactsContract.Data.MIMETYPE, // Type of data (e.g., email or phone)
ContactsContract.Data.DATA1, // The actual data (Phone number / Email string) )
```

```
val results = mutableListOf()
```

```
// Note: The Contact Picker Session Uri doesn't support custom selection & selec-
```

```
launch { contacts = processContactPickerResultUri(resultUri, context) } } } /  
✓ Define the specific contact data fields you need val requestedFields =  
    arrayOf( Email.
```

Android Developers

The Newest 11-Inch Apple iPad Is \$150 Off Ahead of Amazon's Big Spring Sale

Naima Karp · Lifehacker

We may earn a commission from links on this page. Deal pricing and availability subject to change after time of publication.

As part of an early Amazon Big Spring Sale deal, you can get the Apple iPad (11th Gen, 2025) with wifi and cellular capability for \$651 (originally \$799)—its lowest price ever, according to price trackers . Available in four colors, this capable tablet won a PCMag Best of the Year Award in 2025.

2025 Apple iPad - 11-inch (A16)

\$651.20 at Amazon

\$799.00 Save \$147.80

\$651.20 at Amazon

\$799.00 Save \$147.80

The Apple iPad (11th Gen, 2025) comes with 512GB storage and has many of the same specs as its predecessor, with the main difference being a new processor. It has a 5-core CPU A16 for faster performance, and while it doesn't come equipped with Apple's AI tools, it handles everyday tasks like web browsing, streaming media, and playing games with ease—better than most competitors in its class. That said, if you want more advanced processing power, you'll want to upgrade to the pricier iPad Air or iPad Pro , which are also on sale right now.

Design-wise, this iPad is similar to the 2022 model, which is typical for Apple's 11-inch tablets, complete with Touch ID (not Face ID), requiring a fingerprint to unlock. It has a 2,360-by-1,640 resolution, a pixel density of 264ppi, and a 60Hz refresh rate. Compared to the older model, the screen is slightly larger (11 inches vs 10.9 inches), and uses a Liquid Retina display with an LED-backlit panel reaching up to 500 nits of brightness, though it can still show some glare outdoors. Battery life is around 10 hours when using a wifi connection and 9 hours over cellular. The tablet supports Wi-Fi 6 (but not Wi-Fi 6E) as well as Bluetooth 5.3, sub-6GHz 5G, gigabit LTE, and GPS.

It has a 12MP rear camera with an f/1.8 aperture, up to 5x digital zoom, 4K video recording, and support for Apple's Smart HDR 4 processing. The cameras far outperform cheap or similarly priced tablets from other brands. While this isn't Apple's most advanced tablet, it hits the sweet spot in most areas, making it the one that “most people should buy,” according to PCMag, especially at just \$651.20. That said, if you need a more powerful processor, a higher-end display, Wi-Fi 6E, or Apple Intelligence features, you may want to upgrade to the iPad Air , which is also 18% off ahead of Amazon's Big Spring Sale.

Our Best Editor-Vetted Amazon Big Spring Sale Deals Right Now

Apple AirPods Pro 3 Noise Cancelling Heart Rate Wireless Earbuds

— \$199.00

(List Price \$249.00)

Apple iPad 11" 128GB A16 WiFi Tablet (Blue, 2025)

— \$299.00

(List Price \$349.00)

Samsung Galaxy Tab A11+ 128GB Wi-Fi 11" Tablet (Gray)

— \$209.99

(List Price \$249.99)

Sony WH1000XM6- Best Wireless Noise Canceling Headphones

— \$398.00

(List Price \$459.99)

Apple Watch Series 11 (GPS, 42mm, S/M Black Sport Band)

— \$299.00

(List Price \$399.00)

Blink Video Doorbell Wireless (Newest Model) + Sync Module Core

— \$35.99

(List Price \$69.99)

Fire TV Stick 4K Max Streaming Player With Remote

— \$34.99

(List Price \$59.99)

Amazon Kindle Colorsoft 16GB 7" eReader (Black)

— \$169.99

(List Price \$249.99)

Deals are selected by our commerce team

When Rails 7 Doesn't Process application.js

Scott Johnson (fuzzyblog)

This is more than a mildly embarrassing post to have to write but it is an interesting example of how habits can bite you hard – so, so hard.

I recently wanted to start on an application and actually be current on everything: Rails, Bootstrap, etc. I have long admitted to having issues with respect to “modern” web development (specifically JavaScript / the asset pipeline) and I’m mildly starting to deal with those issues. I began with a BootrAils tutorial and started to work through it.

I got all the way to the end and I noticed that I simply could NOT get the popovers to work. I don’t claim to have a great understanding of the asset pipeline but I do know that it is a pre-processor that should get invoked on every page request. I traced the problem back to application.js seemingly not getting processed (a view source didn’t show it). So, just to be sure, I added this to application

Making Urls Paste with Metadata Preview In Facebook, Facebook Messenger, Twitter, Apple Messages

Scott Johnson (fuzzyblog)

I am an old tech guy. I was at the conference where TBL announced the web and I wrote my first HTML snippet just a few years later. But I pretty quickly moved from front end HTML onto back end web development and there are techniques that, while common today, I have never learned.

An example of this is making urls appear, with metadata, when they are pasted into a messaging tool like:

Facebook itself

Twitter itself

We’ve all seen this. Sometimes you paste a url into Facebook Messenger and all that comes out is the url itself. And sometimes you get a picture, a title, etc. The secret to this is three fold:

A set of HTML meta tags for Facebook

A set of HTML meta tags for Twitter

Testing it in a debugging tool like the Facebook Sharing Debugger

Step 1: The HTML Meta Tags for Facebook

Here is an example of the HTML meta tags for Facebook:

Step 2: The HTML Meta Tags for Twitter

Here is an example of the HTML meta tags for Twitter:

Step 3: Testing in The Sharing Debugger

Go to <https://developers.facebook.com/tools/debug/> and paste in your url. You may need to click the Scrape Again button a few times while you work this out.

IN BRIEF

Jeremy Kun (Math n Programming), Jeremy Kun (Math n Programming): And a Pinch of Python Next semester I am a lab TA for an introductory programming course, and it’s taught in Python. My Python experience has a number of gaps in it, so we’ll have the opportunity for a few more Python primers, and small exercises to go along with it. This time, we’ll be investigating the basics of objects and classes, and have some fun with image construction using the Python Imaging Library.

Jeremy Kun (Math n Programming), Jeremy Kun (Math n Programming): So here we are. We have finally made it to a place where we can transition with confidence from the classical continuous Fourier transform to the discrete version, which is the foundation for applications of Fourier analysis to programming. Indeed, we are quite close to unfurling the might of the Fast Fourier Transform algorithm, which efficiently computes the discrete Fourier transform. But because of its focus on algorithmic techniques, we will save it for a main content post and instead focus here on the intuitive connections between the discrete and continuous realms.

Jeremy Kun (Math n Programming), Jeremy Kun (Math n Programming): The Quest to Capture Speed Companies and researchers spend hundreds of millions of dollars for the fruits of their algorithms. Whether one is indexing websites on the internet for search, folding proteins, or figuring out which warehouse is the most cost-effective to ship a product from, improvements in algorithm speed save immense amounts of money. It’s no surprise then, that a similarly immense amount of money has gone into the mathematics behind algorithm analysis.

Jeremy Kun (Math n Programming), Jeremy Kun (Math n Programming): Back when I was first exposed to programming language design, I decided it would be really cool if there were a language that let you define your own number types and then do all your programming within those number types. And since I get excited about math, I think of really exotic number types (Boolean rings, Gaussian integers, Octonions, oh my!). I imagined it would be a language feature, so I could do something like this:

Jeremy Kun (Math n Programming), Jeremy Kun (Math n Programming): Last time we saw a geometric version of the algorithm to add points on elliptic curves. We went quite deep into the formal setting for it (projective space $\mathbb{P}^2$), and we spent a lot of time talking about the right way to define the “zero” object in our elliptic curve so that our issues with vertical lines would disappear, which

Civic Edition

Vol. 1, No. 058 · 2026-03-30

Curated and composed automatically.